

University of Massachusetts Dartmouth  
Department of Computer and Information Science

# CHRONOS

A Masters' Project in  
Computer Science

by

Arundhati Das  
02176805

Submitted in Partial Fulfilment of the  
Requirements for the Degree of  
Master of Science

May 2026

## Abstract

*Chronos* is an educational adventure game built in Unreal Engine 5.6 [9] that serves as a proof of concept for hybrid live AI integration with non-player characters (NPCs) in 3D environments. The game places the player in a fractured timeline, tasking them with traveling through portals into historically reconstructed ancient worlds - including Nabataean Petra (Jordan) and Classical Athens (Greece) - to gather knowledge and collect artifacts that seal a growing temporal rift.

The project initially explored fully AI-generated NPC dialogue using Google Gemini [10], where every interaction was dynamically generated based on structured character profiles and conversation history. While this approach provided high flexibility and immersion, it introduced challenges in token cost, memory usage, latency, and narrative control. To address these limitations, the system was redesigned into a hybrid dialogue architecture.

At its core, *Chronos* demonstrates a hybrid dialogue architecture in which NPCs support both scripted dialogue (pre-authored, indexed lines) and generative AI dialogue powered by Google's Gemini large language model (LLM), toggled per-character via a shared framework. Each AI-driven NPC is backed by a reusable component that issues HTTP requests to the Gemini API at runtime, constructs structured prompts from JSON-defined character templates (personality, era, knowledge boundaries, speech patterns), and maintains persistent conversation history across exchanges. This design enforces historical accuracy - characters such as Socrates, Aspasia, and Nabataean merchants are constrained to knowledge consistent with their time period - while still allowing players to have natural, open-ended conversations with characters.

*Chronos* shows that AI can be added to game characters in a way that works alongside the scripted dialogue, rather than replacing it entirely. It uses a flexible, data-driven setup where new historically based characters can be added using simple configuration files. This combined approach made it much easier to build characters that can hold real conversations with players - without developers having to manually write out every possible thing a character might say. The AI makes the gameplay interactive through things like a dynamic hint system, where players can ask a character for help during a quiz and get a response that fits that character's personality and time period. On top of the player experience, AI also helped during the creative process itself - making it easier to figure out how each character should speak and what kinds of historically grounded stories they might be part of, all through prompt engineering rather than writing everything from scratch. Altogether, *Chronos* offers a practical, ready-to-reuse framework for developers and researchers who want to bring large language models into educational games in a way that is structured and makes real sense within the game.

# Table of Contents

|                                                               |    |
|---------------------------------------------------------------|----|
| Abstract .....                                                | 2  |
| Table of Contents .....                                       | 3  |
| List of Figures: .....                                        | 6  |
| List of Tables:.....                                          | 7  |
| Chapter 1: Introduction.....                                  | 8  |
| 1.1 Background .....                                          | 8  |
| 1.2 Problem Statement .....                                   | 8  |
| 1.3 Project Objectives .....                                  | 8  |
| 1.4 Scope.....                                                | 9  |
| 1.5 Target Audience.....                                      | 9  |
| 1.6 Core Pillars.....                                         | 9  |
| Chapter 2: Related Work and Current Landscape .....           | 11 |
| 2.1 AI in Educational Games Before Large Language Models..... | 11 |
| 2.3 The Emergence of Large Language Models in Games .....     | 11 |
| 2.4 How Chronos explores the LLM in Games.....                | 11 |
| Chapter 3: System Design & Architecture .....                 | 13 |
| 3.1 Overview .....                                            | 13 |
| 3.2 Quest System.....                                         | 13 |
| 3.2.1 Data Structure.....                                     | 13 |
| 3.2.2 Runtime Architecture .....                              | 14 |
| 3.2.3 Objective Completion.....                               | 14 |
| 3.3 Dialogue System.....                                      | 14 |
| 3.3.1 NPC Base Class.....                                     | 14 |
| 3.3.2 Scripted Dialogue Flow .....                            | 15 |
| 3.3.3 AI Dialogue Integration.....                            | 15 |
| 3.3.4 AI Hint System.....                                     | 15 |
| 3.4 Knowledge Points System.....                              | 15 |
| 3.5 Codex System.....                                         | 16 |
| 3.6 Inventory System.....                                     | 16 |
| 3.7 Save and Persistence System.....                          | 16 |
| 3.7.1 Level Transition Persistence .....                      | 17 |
| 3.7.2 Save Game Persistence.....                              | 17 |
| 3.8 UI Architecture.....                                      | 17 |
| Chapter 4: Quest System Design and Implementation .....       | 18 |

|                                                         |    |
|---------------------------------------------------------|----|
| 4.1 Quest Content Design.....                           | 18 |
| 4.1.1 Petra Quest Line .....                            | 20 |
| 4.1.2 Athens Quest Line .....                           | 20 |
| 4.2 Quest Chaining in Practice .....                    | 21 |
| Chapter 5: Dialogue System and AI Integration .....     | 22 |
| 5.1 Dialogue Widget.....                                | 22 |
| 5.2 HTTP Client Component.....                          | 22 |
| 5.2.1 Component Architecture: .....                     | 22 |
| 5.2.2 API Configuration .....                           | 23 |
| 5.2.3 Character Persona System .....                    | 23 |
| 5.2.4 Prompt Construction.....                          | 24 |
| 5.2.5 Delegate Binding .....                            | 26 |
| 5.2.6 Historical Accuracy Constraints.....              | 26 |
| 5.3 Evolution of current Dialogue System: .....         | 26 |
| 5.3.1 Initial Approach - Fully AI-Driven NPCs .....     | 26 |
| 5.3.2 The Hybrid Solution .....                         | 27 |
| 5.3.3 Conversation History and Context Management.....  | 27 |
| 5.3.4 Token Optimisation.....                           | 27 |
| Chapter 6: Codex and Inventory Systems .....            | 28 |
| 6.1 Overview .....                                      | 28 |
| 6.2 Codex System.....                                   | 28 |
| 6.2.1 Purpose .....                                     | 28 |
| 6.2.2 Implementation.....                               | 28 |
| 6.2.3 Entry Data.....                                   | 29 |
| 6.2.4 Unlock Mechanisms .....                           | 29 |
| 6.2.5 Codex Entries in Chronos.....                     | 29 |
| Table 9: <i>Codex Entries — Athens Quest Line</i> ..... | 29 |
| 6.2.6 Codex UI .....                                    | 30 |
| 6.3 Inventory System.....                               | 30 |
| 6.3.1 Purpose .....                                     | 30 |
| 6.3.2 Implementation.....                               | 30 |
| 6.3.3 Item Actors .....                                 | 31 |
| 6.3.4 Item Usage in Dialogue .....                      | 31 |
| 6.3.5 Inventory UI .....                                | 31 |
| 6.4 Cross-Level Persistence.....                        | 31 |

|                                                          |    |
|----------------------------------------------------------|----|
| Chapter 7: Expansion Guide.....                          | 32 |
| 7.1 Purpose .....                                        | 32 |
| 7.2 Adding a New World.....                              | 32 |
| 7.3 Quick Reference .....                                | 32 |
| 7.4 Important Notes.....                                 | 33 |
| 7.5 AI NPCs in the Game:.....                            | 33 |
| Petra World — Nabataean Kingdom (~1st Century AD): ..... | 33 |
| Athens World — Classical Athens (~5th Century BC):.....  | 34 |
| Chapter 8 Testing Phase: .....                           | 34 |
| 8.1 AI Integration Testing via Postman.....              | 34 |
| 8.1.1 Character Persona Testing .....                    | 34 |
| Chapter 9: Conclusion and Future Work.....               | 35 |
| 9.1 Summary .....                                        | 35 |
| 9.2 Limitations.....                                     | 35 |
| 9.3 Future Work .....                                    | 36 |
| References: .....                                        | 36 |
| Appendix: .....                                          | 37 |
| A: Asset Credits.....                                    | 37 |
| B: Game Screenshots.....                                 | 37 |
| B.1 Petra World: .....                                   | 40 |
| B.2 Athens World:.....                                   | 46 |

## List of Figures:

|                                                             |    |
|-------------------------------------------------------------|----|
| Figure 1: High-Level System Architecture Overview           | 13 |
| Figure 2: Quest Data Structure Hierarchy                    | 13 |
| Figure 3: Objective Completion Flow                         | 14 |
| Figure 4: Quest Content Design Overview                     | 19 |
| Figure 5: Quest Chaining Flow — Petra and Athens Lines      | 21 |
| Figure 6: Gemini API — JSON Request Body Template           | 24 |
| Figure 7: Prompt Construction and AI Request Flow           | 26 |
| Figure 8: Codex and Inventory Systems Overview              | 29 |
| Figure 9: Inventory System — Item Interaction Concept       | 31 |
| Figure 10: Game Screenshots — Title and Menu Screens        | 39 |
| Figure 11: Void — World Environments                        | 40 |
| Figure 12: Game Screenshots — UI and Gameplay (continued)   | 41 |
| Figure 13: Petra World — Environment                        | 41 |
| Figure 14: Petra World — Al-Khazneh                         | 42 |
| Figure 15: Petra World — Merchant Amina                     | 42 |
| Figure 16: Petra World — Amina Teaching Phase               | 42 |
| Figure 17: Petra World — Amina AI Response                  | 43 |
| Figure 18: Petra World — KP notifications                   | 43 |
| Figure 18: Petra World — Obodas                             | 44 |
| Figure 19: Inspect Item Feature                             | 44 |
| Figure 20: Khalil Stopping the Player                       | 44 |
| Figure 21: Petra World — Khalil                             | 46 |
| Figure 22: Petra World — Talking to Khalil                  | 46 |
| Figure 23: Petra World — Talking to Khalil Continued        | 47 |
| Figure 24: Petra World — Treasury Artifact                  | 47 |
| Figure 25: Petra World — Complete Treasury Mission          | 47 |
| Figure 26: Athens World — Parthenon                         | 48 |
| Figure 27: Athens World — Aspasia                           | 48 |
| Figure 26: Athens World — Aspasia Teaching Phase            | 48 |
| Figure 27: Athens World — Talking to Aspasia                | 49 |
| Figure 28: Athens World — Aspasia Quiz                      | 49 |
| Figure 29: Athens World — Completing mission                | 49 |
| Figure 30: Athens World — Socrates                          | 50 |
| Figure 31: Athens World — Talking to Socrates               | 50 |
| Figure 32: Athens World — Socrates Quiz                     | 50 |
| Figure 33: Athens World - AI response from Socrates         | 51 |
| Figure 34: Athens World — Statue of Athena inside Parthenon | 51 |
| Figure 35: Athens World — Collecting the Golden Owl         | 51 |
| Figure 36: Athens World — Returning the Golden Owl          | 52 |
| Figure 37: Void Final Mission to Complete the Game          | 52 |
| Figure 38: Game Complete and Game Over                      | 52 |

## List of Tables:

|                                                             |    |
|-------------------------------------------------------------|----|
| Table 1: Knowledge Points - Actions and Rewards             | 16 |
| Table 2: Level Transition — Saved Data via Game Instance    | 17 |
| Table 3: UI Widget Blueprints Overview                      | 18 |
| Table 4: The Treasury's Secret - Quest Stages and Mechanics | 20 |
| Table 5: HTTP Client Component — Configuration Properties   | 24 |
| Table 6: Character Persona — JSON Field Definitions         | 25 |
| Table 7: Base Prompt Template Structure                     | 25 |
| Table 8: BP_CodexSystem — Component Properties              | 30 |
| Table 9: Codex Entries — Athens Quest Line                  | 31 |
| Table 10: BP_InventoryComponent — Component Properties      | 31 |
| Table 11: Item Actor Blueprints                             | 32 |
| Table 12: Expansion Quick Reference Checklist               | 34 |
| Table 13: AI Character Persona — Testing Criteria           | 35 |
| Table 14: Evaluation Against Project Objectives             | 36 |
| Table 15: 3D Environment Asset Credits                      | 38 |
| Table 16: Character Asset Credits                           | 38 |
| Table 17: Other Asset Credits                               | 38 |

# Chapter 1: Introduction

## 1.1 Background

The teaching of ancient history in traditional educational settings often relies on textbooks, lectures, and static visual aids. While these methods convey factual information, they frequently fail to engage learners in a manner that promotes deep understanding and long-term retention. Students read about civilisations such as Nabataean Petra or Classical Athens without ever experiencing the spatial, social, or cultural dimensions of these worlds. This disconnect between content delivery, and meaningful engagement represents a significant challenge in history education.

In recent years, video games have emerged as a compelling medium for educational purposes. Unlike passive learning materials, games place the learner in an active role - requiring them to make decisions, solve problems, and engage with content in order to progress. The interactive nature of games aligns closely with constructivist learning theory, which holds that knowledge is most effectively constructed through experience and active participation rather than passive reception.

## 1.2 Problem Statement

Despite the growing body of research supporting game-based learning, the majority of commercially available educational games targeting historical content suffer from one or more of the following limitations: superficial engagement with the subject matter, a lack of meaningful consequence for incorrect knowledge, absence of knowledge verification mechanisms, and limited narrative depth. Many educational games treat learning as a secondary feature layered on top of generic gameplay, rather than integrating it as the core mechanic that drives progression.

Furthermore, only few educational games leverage modern AI capabilities to create dynamic, conversational learning experiences. Most rely entirely on pre-scripted dialogue trees, which limits the depth of player inquiry and reduces the sense of interacting with a knowledgeable character.

## 1.3 Project Objectives

Chronos was developed to address these gaps. The primary objectives of the project are as follows:

- To design and develop an educational video game that teaches players about the history and culture of Nabataean Petra and Classical Athens through interactive gameplay.
- To implement a quest-based learning system that follows a Teach–Test–Advance pedagogical loop, ensuring that players must retain and apply knowledge in order to progress.
- To integrate AI-augmented NPC dialogue alongside scripted educational content, enabling players to engage in free-form conversation with historically contextualised characters.
- To create a consequence-driven Knowledge Points system that introduces meaningful stakes for correct and incorrect responses during knowledge verification phases.
- To develop a persistent Codex system that records the player's educational progress as a referenceable encyclopaedia.
- To build the game in Unreal Engine 5, employing a modular and scalable architecture capable of supporting additional historical worlds in future iterations.

## 1.4 Scope

Chronos is a third-person 3D educational game comprising three interconnected environments: a hub world referred to as the Void, and two historical worlds - Petra and Athens. The Petra world covers Nabataean trade, architecture, water engineering, and the Treasury of Al-Khazneh through interactions with four NPCs across a multi-stage quest chain. The Athens world covers the Parthenon, the goddess Athena, Socratic philosophy, and a narrative involving the theft and recovery of a sacred golden owl through interactions with two NPCs across four linked quests. The game features a hybrid dialogue system combining scripted content with AI-generated responses, an inventory system for item-based problem solving, a Knowledge Points mechanic with Game Over consequences, an XP and levelling system, a Codex encyclopaedia, and a save system that preserves all player state across level transitions and sessions.

## 1.5 Target Audience

The game is designed for secondary school and undergraduate students studying ancient history, as well as general audiences with an interest in historical education through interactive media. The third-person perspective, quest-driven structure, and conversational gameplay are intended to be accessible to players with varying levels of gaming experience.

## 1.6 Core Pillars

Chronos is an educational video game built in Unreal Engine 5 that teaches players about ancient civilisations through interactive gameplay framed within a time-travel narrative. The design is grounded in the following core pillars.

- **Education Through Exploration:** Learning in Chronos occurs through direct engagement rather than passive reading. Players travel to historically authentic recreations of Nabataean Petra and Classical Athens, where educational content is delivered through structured NPC conversations. Every quest objective maps directly to a learning outcome - from Nabataean trade routes and desert water engineering in Petra, to Socratic philosophy and the Panathenaic festival in Athens.
- **Narrative-Driven Time Travel:** A fractured rift between worlds serves as the narrative framework. A mysterious Stranger in the Void — a hub world between dimensions - sends the player through portals into ancient civilisations to gather knowledge and recover artefacts. This framing provides narrative urgency while offering structural flexibility to span multiple historical periods without requiring a direct connection between them.
- **Teach–Test–Advance Loop:** The quest system follows a three-phase pedagogical loop. NPCs first deliver historically accurate content through branching dialogue. Players are then tested through multiple-choice quizzes drawn from the material just covered. Successful completion unlocks the next quest and advances the narrative. This ensures players must retain and apply what they have learned in order to progress.
- **AI-Augmented Dialogue:** Chronos employs a hybrid dialogue system combining scripted educational content with AI-generated free-form responses. Players can type questions to NPCs and receive contextually appropriate answers guided by character-specific prompts. An AI hint system during quizzes provides a safety net at the cost of Knowledge Points, encouraging self-reliance.

- **The Codex System:** Completed learning objectives permanently unlock entries in an in-game Codex — a categorised encyclopaedia that persists across levels. It serves as both a reference tool during gameplay and a record of the player's educational progress.
- **Dual-World Structure:** Two complete historical worlds are available: Petra, featuring NPCs such as the merchant Obodas and the Treasury guard Khalil, covering trade, architecture, and Nabataean culture; and Athens, featuring the priestess Aspasia and Socrates, covering philosophy, mythology, and the mystery of a stolen sacred artefact. Players may complete the worlds in any order.
- **Knowledge Points as Consequence:** Players earn Knowledge Points for correct quiz answers and lose them for incorrect responses or requesting hints. Reaching zero triggers a Game Over. This creates meaningful stakes for the educational content and incentivises attentive engagement during teaching phases.
- **Inventory and Item-Based Problem Solving:** Physical item interaction complements the dialogue system. Players locate and use items such as honey cakes to bribe a guard, recover the golden owl from within the Parthenon, and discover artefacts inside Al-Khazneh — adding an exploratory dimension beyond conversation.
- **Player Progression and Persistence:** An experience point and levelling system tracks completion progress alongside the Knowledge Points mechanic. A Game Instance-based persistence system preserves all player state - quests, inventory, codex entries, KP, and XP - across level transitions and play sessions, ensuring educational progress is never lost.

## Chapter 2: Related Work and Current Landscape

### 2.1 AI in Educational Games Before Large Language Models

The use of artificial intelligence in educational software predates modern large language models by several decades.

Within games specifically, AI-driven NPCs have historically relied on finite state machines, behaviour trees, and pre-authored dialogue trees. Games such as *Faade* [3] attempted to create believable conversational agents using a drama management system, but the scope of recognisable player input remained limited. Educational games that employed conversational NPCs, such as those built on the ORIENT framework or the Crystal Island platform [4], used scripted branching narratives with limited natural language understanding. The NPCs could respond to anticipated inputs but could not handle unanticipated questions or engage in genuinely open-ended conversation. The fundamental limitation of pre-LLM AI in educational games was rigidity. NPCs could deliver pre-authored educational content effectively, but they could not adapt to the specific curiosity of an individual learner. A student who wanted to ask a follow-up question not anticipated by the dialogue tree would receive no meaningful response. This constrained the sense of immersion and limited the educational depth of NPC interactions to what the developers had explicitly scripted.

### 2.3 The Emergence of Large Language Models in Games

The release of large language models such as GPT-3 [5] and subsequent models fundamentally changed the possibilities for NPC dialogue in games. LLMs can generate contextually appropriate, coherent natural language responses to arbitrary player input, enabling NPCs to engage in open-ended conversation for the first time. Several projects have begun exploring this capability.

NVIDIA's ACE (Avatar Cloud Engine) technology [6], demonstrated in 2023, showcased NPCs powered by LLMs that could hold unscripted conversations with players in real time. Inworld AI [7] has developed a middleware platform that allows game developers to create LLM-driven characters with defined personalities, knowledge boundaries, and emotional states. The independently developed game *Suck Up!* [8] uses LLM-powered NPCs as its central mechanic, requiring players to convince AI characters to invite them inside through persuasive conversation.

However, the application of LLMs in specifically educational games remains largely unexplored. Most LLM-NPC demonstrations focus on entertainment, narrative emergence, or social simulation rather than structured pedagogy. The challenge lies in combining the flexibility of LLM-generated dialogue with the rigour required for educational content delivery - ensuring that AI responses are historically accurate, pedagogically appropriate, and integrated with game progression systems.

### 2.4 How Chronos explores the LLM in Games

Chronos addresses the limitations identified in the existing landscape through a hybrid dialogue architecture that combines scripted educational content with LLM-augmented free-form conversation. The scripted layer ensures that core historical content is delivered accurately and that quest progression is tied to specific learning objectives through a Teach-Test-Advance loop. The AI layer, powered by an LLM accessed via an HTTP client component, allows players to ask NPCs open-ended questions beyond the scripted paths - for instance, asking Socrates a

philosophical question not covered by the predefined dialogue options, or requesting a hint during a quiz in exchange for Knowledge Points.

Each NPC in Chronos is assigned a character-specific system prompt that constrains the LLM's responses to the historical period, personality, and knowledge domain of that character. Aspasia responds as a priestess of Athena with knowledge of the Parthenon and the Panathenaic festival. Socrates responds as a philosopher who questions assumptions and speaks in paradoxes. This approach maintains narrative immersion while leveraging the generative capabilities of modern language models.

Critically, Chronos differs from existing educational games in three respects. The AI dialogue layer enables a depth of player inquiry that scripted dialogue trees alone cannot provide, while the scripted layer ensures pedagogical structure that pure LLM conversation cannot guarantee. This hybrid approach represents a synthesis of the structured rigour of traditional educational game design with the conversational flexibility enabled by contemporary large language models.

## Chapter 3: System Design & Architecture

### 3.1 Overview

Chronos is built in Unreal Engine 5.6 using a Blueprint-driven architecture. The game comprises three levels - the Void hub world, the Petra world, and the Athens world - connected through a portal system that preserves player state across level transitions via a Game Instance. The core systems include a data-driven quest system, a hybrid scripted and AI-powered dialogue system, a Knowledge Points mechanic, a Codex encyclopaedia, an inventory system, and a persistence layer. This chapter describes the design and interconnection of each system.

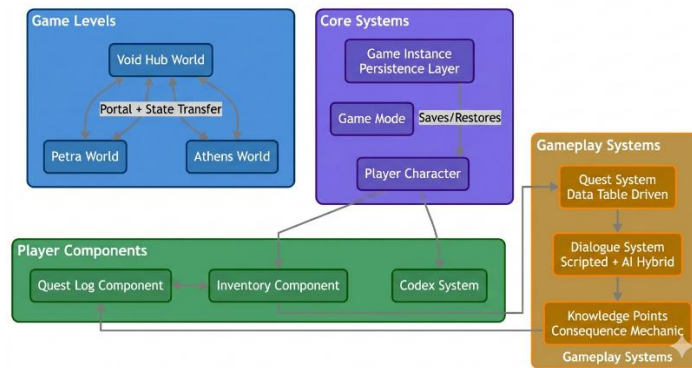


Figure 1: High-Level System Architecture Overview

### 3.2 Quest System

The quest system is the primary driver of game progression. It is built on a data-driven architecture centred on a Data Table (DT\_QuestList) that defines all quests, their stages, and their objectives.

#### 3.2.1 Data Structure

Each quest is defined by the following hierarchy:

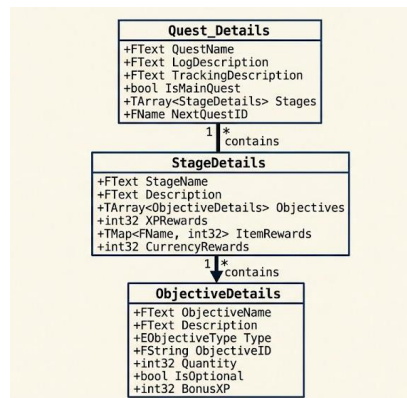


Figure 2: Quest Data Structure Hierarchy

A quest may contain multiple stages, each of which contains one or more objectives. When all objectives within a stage are completed, the quest automatically advances to the next stage. When the final stage is completed, the

quest is marked as complete and the next quest in the chain is added to the player's active quest list, as defined by the NextQuestID field.

### 3.2.2 Runtime Architecture

At runtime, the quest system operates through three key components:

- **BP\_QuestBase:** An Actor class spawned for each active quest. It holds the quest's runtime state including current stage, objective progress, and completion status. It provides functions for updating objective progress, checking stage completion, and advancing stages.
- **QuestLog\_Component:** An Actor Component attached to the player character. It maintains arrays of active quest IDs, active quest actors, completed quest IDs, and completed quest actors. It provides functions for adding new quests, completing quests, and querying active quest status.
- **DT\_QuestList:** The Data Table that stores all quest definitions. When a new quest is added, BP\_QuestBase reads its definition from this table and initialises its state accordingly.

### 3.2.3 Objective Completion

Objectives are completed through calls to UpdateObjectiveProgress, which can originate from NPC dialogue events, item pickups, trigger overlaps, or any other gameplay interaction. The function locates the matching objective by its string ID, increments its progress, and checks whether the stage is now complete. This string-based matching system allows any Blueprint in the project to complete any objective without requiring a direct reference to the quest actor.

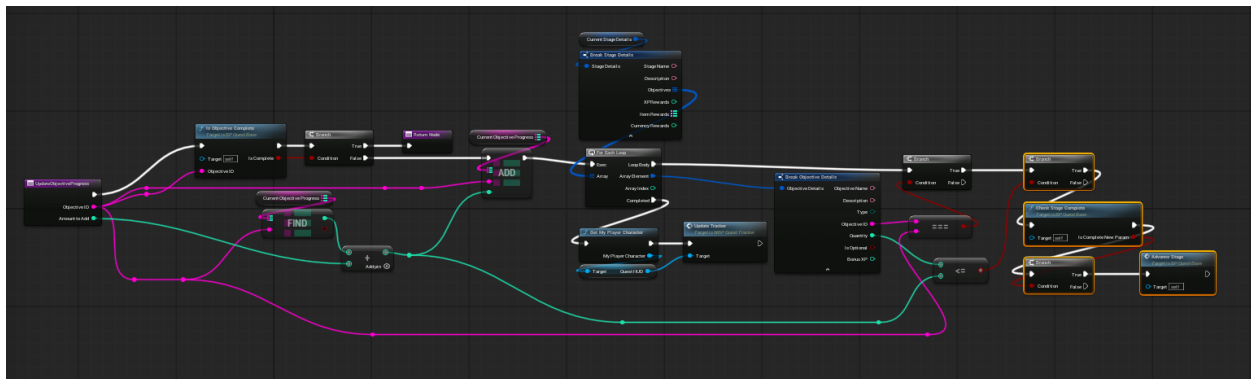


Figure 3: Objective Completion Flow

## 3.3 Dialogue System

The dialogue system is built on a base class (BP\_NPCBase) that all NPC characters inherit from. It supports two modes of operation: scripted dialogue with branching options, and AI-generated free-form conversation.

### 3.3.1 NPC Base Class

BP\_NPCBase extends the Character class and provides the following shared infrastructure:

- **Dialogue array:** A predefined array of strings for sequential non-branching dialogue.
- **Talk Index:** Tracks the current position in the dialogue array.
- **UI reference:** A reference to the dialogue widget (WB\_Dialogue) created when conversation begins.
- **Conversation History:** An array of chat messages maintaining the full conversation transcript.

- **DialogueKeywords:** A map of keywords to objective IDs, used to detect when AI responses cover educational topics.
- **ObjectivesToComplete:** An array of objective IDs that this NPC is responsible for completing.
- **QuestToCodexMap:** A map linking objective IDs to Codex entry IDs for automatic unlocking.

### 3.3.2 Scripted Dialogue Flow

Each NPC child class implements a `DialogueStage` variable that controls which phase of conversation the player is in. The `Talk` event checks the current stage and presents the appropriate greeting, dialogue options, and quiz questions. The `ReceivePlayerInput` event handles the player's selection, delivers the NPC's response, completes relevant objectives, unlocks Codex entries, and advances the dialogue stage.

The following diagram illustrates the Teach–Test–Advance loop as implemented in the dialogue system:

### 3.3.3 AI Dialogue Integration

When an NPC has `Use AI Dialogue` enabled, the system integrates with an external LLM via `BP_HttpClient_Component`. The AI integration operates as follows:

1. The player types a free-form message in the dialogue input box.
2. The message is added to the conversation history and a follow-up prompt is constructed using `Build_Followup_Prompt`, which includes the character ID (e.g., "socrates", "aspasia") to maintain persona consistency.
3. The prompt is sent to the AI backend via `Send AI Request`.
4. When the response is received, the `OnAIResponseReceived_Event` fires, adding the response to the conversation history and displaying it in the dialogue widget.
5. The `CheckQuestKeywords` function scans the AI response for predefined keywords mapped to quest objectives. If a keyword is detected, the corresponding objective is automatically completed.

This architecture allows NPCs to answer questions beyond their scripted dialogue paths while still contributing to quest progression through keyword detection.

### 3.3.4 AI Hint System

During quiz phases, a fourth dialogue option - "Ask [NPC Name] for a hint" - is presented alongside the three quiz answers. Selecting this option hides the quiz options, reveals the text input box, and sets the `IsAwaitingHintInput` flag. The player's typed question is prepended with context about the current quiz question and sent to the AI backend. Upon receiving the AI response, the system deducts 5 KP as a cost for the hint, then re-displays the quiz options. This mechanic encourages players to attempt answers independently while providing a safety net for those who are struggling.

## 3.4 Knowledge Points System

Because KP is both the reward for learning and a resource that can be spent on AI hints, every quiz becomes a small risk/reward decision. Answering from memory is efficient; asking the AI for help costs KP; answering wrongly costs more. The economy is tuned per-NPC - Khalil's treasury quiz carries -10 for wrong answers, while Amina's introduction quiz only costs -3. Higher-stakes content is genuinely higher-stakes. Knowledge Points (KP) serve as the game's consequence mechanic, providing tangible stakes for the educational content.

| Action                | KP Effect |
|-----------------------|-----------|
| Correct quiz answer   | +10       |
| Incorrect quiz answer | -5        |
| Requesting an AI hint | -5        |
| KP reaches zero       | Game Over |

Table 1: Knowledge Points - Actions and Rewards

KP is managed through the player character's AddKP and DeductKP functions. The DeductKP function returns a boolean indicating whether a Game Over has been triggered, which the calling system uses to display the Game Over widget. KP is displayed on the HUD and persists across level transitions via the Game Instance.

### 3.5 Codex System

The Codex is implemented as an Actor Component (BP\_CodexSystem) attached to the player character. It maintains an array of unlocked entry IDs (UnlockedEntries) and reads entry data from a Data Table (DT\_CodexEntries).

Entries are unlocked through two mechanisms:

1. Direct unlock: NPC dialogue scripts call UnlockEntry with a specific entry ID after completing a learning objective.
2. Quest-to-Codex mapping: Each NPC maintains a QuestToCodexMap that maps objective IDs to Codex entry IDs, enabling automatic unlocking when objectives are completed.

The Codex UI (WBP\_CodexJournal\_V2) displays unlocked entries in a categorised journal format, allowing players to review learned material at any time.

### 3.6 Inventory System

The inventory system is implemented as an Actor Component (BP\_InventoryComponent) attached to the player character. It maintains an array of collected item IDs and provides functions for adding items, removing items, checking whether an item has been collected, and querying the inventory contents.

Items are collected through interaction with item actors in the world (e.g., BP\_GoldenOwl, BP\_HoneyCake, BP\_TreasuryArtifact). Item usage is handled through NPC dialogue logic - for example, when the player selects "I found it! Here, take it back" during conversation with Aspasia, the system checks whether golden\_owl is present in the inventory via IsItemCollected, removes it via RemoveItem, and completes the corresponding quest objective.

### 3.7 Save and Persistence System

Chronos employs a two-tier persistence system to handle both level transitions and save/load functionality.

### 3.7.1 Level Transition Persistence

When the player transitions between levels via a portal, the level changer Blueprint saves the following data to the Game Instance (BP\_GameInstance):

| Data                 | Source             | Game Instance Variable     |
|----------------------|--------------------|----------------------------|
| Knowledge Points     | Player Character   | Saved_KnowledgePoints      |
| Player Level         | Player Character   | Saved_PlayerLevel          |
| Current XP           | Player Character   | Saved_CurrentXP            |
| XP to Next Level     | Player Character   | Saved_XPToNextLevel        |
| Max Knowledge Points | Player Character   | Saved_MaxKnowledgePoints   |
| Active Quest IDs     | QuestLog_Component | Saved_ActiveQuests         |
| Completed Quest IDs  | QuestLog_Component | Saved_CompletedQuests      |
| Inventory Items      | InventoryComponent | Saved_InventoryItems       |
| Codex Entries        | CodexSystem        | Saved_UnlockedCodexEntries |

Table 2: Level Transition — Saved Data via Game Instance

On the destination level, the player character's BeginPlay event detects the presence of saved data in the Game Instance and restores all values, re-adding active quests, repopulating the inventory, and re-unlocking Codex entries.

### 3.7.2 Save Game Persistence

For persistence across play sessions, the game uses a BP\_SaveGame object that serialises the same data to disk. The Game Instance provides SaveGameFunction and LoadtheGameFunction to handle writing and reading save data. The player character's BeginPlay checks whether a save game load has been requested and restores state accordingly.

## 3.8 UI Architecture

The game's user interface comprises several interconnected widget Blueprints:

| Widget               | Purpose                                                                                          |
|----------------------|--------------------------------------------------------------------------------------------------|
| WB_Dialogue          | Main dialogue interface — displays chat lines, dialogue options, text input box, and hint button |
| WBP_QuestTracker     | HUD element displaying the currently tracked quest and its objectives                            |
| WBP_QuestJournal     | Full-screen journal listing all active and completed quests with details                         |
| WBP_CodexJournal_V2  | Categorised encyclopaedia displaying unlocked Codex entries                                      |
| WBP_InventoryJournal | Inventory screen displaying collected items                                                      |

| Widget                          | Purpose                                                         |
|---------------------------------|-----------------------------------------------------------------|
| WBP_KPNotification              | Popup notification for KP gains and losses                      |
| WBP_MissionCompleteNotification | Notification displayed when a quest stage or quest is completed |
| WBP_GameOver                    | Game Over screen displayed when KP reaches zero                 |
| WBP_GameComplete                | End-of-game screen displayed upon completing both worlds        |
| WBP_PauseMenu                   | Pause menu with resume, save, and quit options                  |
| WBP_LoadingScreen               | Loading screen displayed during level transitions               |

*Table 3: UI Widget Blueprints Overview*

The dialogue widget (WB\_Dialogue) is the most complex UI element. It is created dynamically when the player initiates conversation with an NPC, receives a reference to the linked NPC actor, and is destroyed when the conversation ends. It supports both option-based selection (for scripted dialogue and quizzes) and free-text input (for AI conversations and hints).

## Chapter 4: Quest System Design and Implementation

### 4.1 Quest Content Design

This section details the specific quest content implemented in Chronos, illustrating how the data-driven architecture described in the System Design chapter was applied to create two complete educational quest lines.

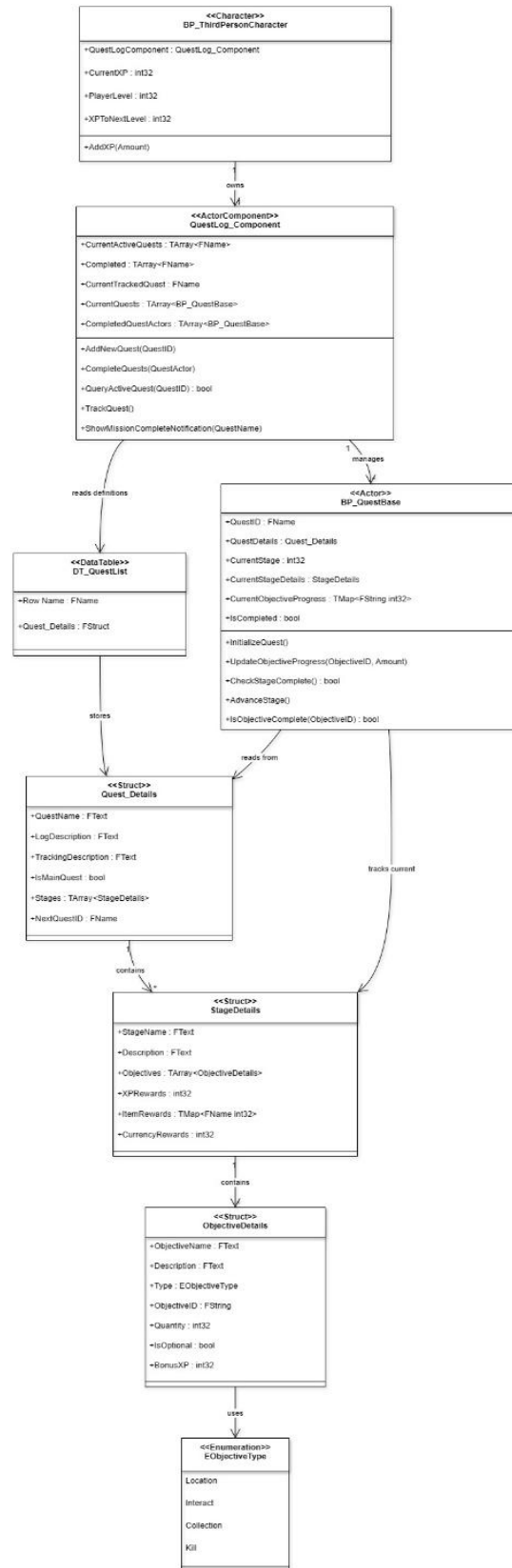


Figure 4: Quest Content Design Overview

#### 4.1.1 Petra Quest Line

The Petra quest line comprises four quests totalling nine stages and seventeen objectives. It follows a narrative arc from arrival and introduction, through learning and investigation, to puzzle-solving and discovery.

The introductory quest ("Welcome to Petra") establishes the setting through conversation with Merchant Amina. Four learning objectives cover the city's identity, its trade significance, and the tasks ahead, followed by a completion objective that gates progression. Each objective is completed when the player selects the corresponding dialogue option from Amina's scripted conversation.

The merchant quest ("Secrets of the Merchant") deepens the educational content through Obodas, who teaches five topics: the frankincense trade, the Nabataean tax system, the purpose of Al-Khazneh, desert water engineering, and the bitumen trade with Rome. These topics were selected to cover the economic, architectural, and engineering achievements of the Nabataean civilisation.

The Treasury quest ("The Treasury's Secret") is the most complex quest in the game, spanning six stages that introduce gameplay mechanics beyond dialogue:

| Stage | Name                  | Mechanic Introduced                          |
|-------|-----------------------|----------------------------------------------|
| 1     | The Locked Door       | NPC interaction as gatekeeping               |
| 2     | Gathering Intel       | Information gathering from secondary NPCs    |
| 3     | Get the Honey Cake    | Item acquisition                             |
| 4     | Past the Guard        | Item-based problem solving (bribery)         |
| 5     | Khalil's Lesson       | Knowledge verification quiz                  |
| 6     | The Treasury's Secret | Environmental exploration and item discovery |

*Table 4: The Treasury's Secret - Quest Stages and Mechanics*

This progressive introduction of mechanics ensures the player is not overwhelmed while maintaining educational engagement throughout.

#### 4.1.2 Athens Quest Line

The Athens quest line comprises four quests totalling four stages and fourteen objectives. It follows a narrative arc centred on the mystery of a stolen sacred artefact.

The introductory quest ("Welcome to Athens") mirrors the Petra introduction in structure. Five learning objectives cover the Parthenon, Athena, the Panathenaic festival, the golden owl, and the theft - each completed through dialogue with Aspasia.

The Socrates quest ("The Wisdom of Socrates") is the most objective-dense quest in the game, with six objectives in a single stage. It combines the teaching phase (finding Socrates, learning about ethics, knowledge, Kleon's movements, and the trial) with a verification phase (passing the Socratic test). The single-stage design reflects the conversational nature of the encounter - the player engages in an extended dialogue rather than moving through distinct physical locations.

The investigation quest ("The Missing Owl") shifts gameplay to environmental exploration. A single Collection-type objective requires the player to physically locate the golden owl behind the statue of Athena inside the Parthenon. This represents a deliberate change of pace from dialogue-heavy quests. The return quest ("Return of the Owl") brings the narrative full circle by requiring the player to return the recovered owl to Aspasia through an inventory check, completing the Athens storyline.

## 4.2 Quest Chaining in Practice

The chain operates automatically through the AdvanceStage function - when the final stage of a quest completes, NextQuestID is read and passed to AddNewQuest. The only exception is MainQuestFinal, which is added dynamically by BP\_Stranger\_Void upon detecting that both return-to-void quests exist in the Completed array. This dynamic addition is necessary because the final quest depends on two independent chains rather than a single linear predecessor.

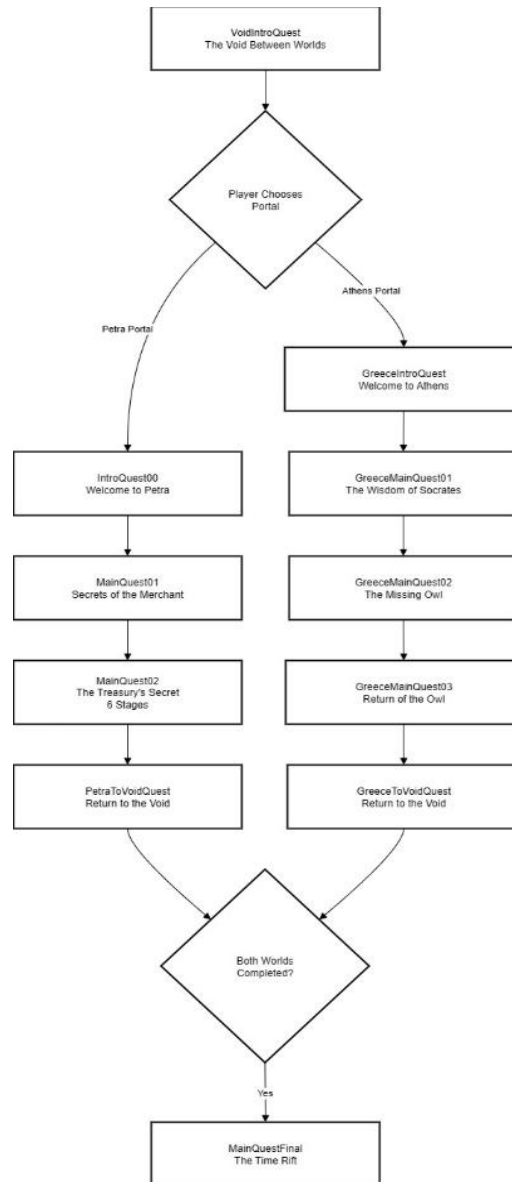


Figure 5: Quest Chaining Flow — Petra and Athens Lines

## Chapter 5: Dialogue System and AI Integration

The dialogue system in Chronos serves as the primary interface between the player and the educational content. Unlike traditional game dialogue systems that rely solely on pre-authored branching trees, Chronos employs a hybrid architecture that combines scripted educational dialogue with AI-generated free-form responses powered by Google's Gemini large language model. This chapter details the implementation of both layers and how they interact to deliver a structured yet flexible learning experience.

### 5.1 Dialogue Widget

All NPC conversations are presented through a single reusable widget, `WB_Dialogue`, which is created dynamically when the player initiates conversation and destroyed when the conversation ends. The widget comprises the following elements:

- **Chat History:** A scrollable vertical box displaying the conversation transcript as sequential chat lines, with visual distinction between NPC and player messages.
- **NPC Response Text:** A text block displaying the current NPC statement.
- **Options Panel:** A vertical box containing up to four buttons for dialogue option selection. Three buttons are used for standard dialogue choices and quiz answers. A fourth button, hidden by default, is revealed during quiz phases for the AI hint option.
- **Input Box:** An editable text field, hidden by default, that appears when the player is in free-form AI conversation mode or requesting a hint. It accepts typed input and submits it on Enter.

The widget receives a reference to the linked NPC actor upon creation, enabling it to call NPC functions directly when the player interacts with UI elements. When a dialogue option is selected, the widget calls `OnOptionSelected`, which forwards the selected text to the NPC's `ReceivePlayerInput` event. When the player types in the input box and presses Enter, the text is similarly forwarded.

### 5.2 HTTP Client Component

The `BP_HttpClient_Component` is the core technical component that enables AI-powered NPC dialogue in Chronos. It is an Actor Component inherited by all NPCs through `BP_NPCBase` and handles the full lifecycle of communication with Google's Gemini large language model API.

#### 5.2.1 Component Architecture:

The component maintains the following configuration properties:

| Property                          | Type                 | Purpose                                                                                                                      |
|-----------------------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>API_Base_URL</code>         | <code>FString</code> | The Gemini API endpoint<br>( <code>generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent</code> ) |
| <code>API_Key</code>              | <code>FString</code> | Authentication key for API access                                                                                            |
| <code>JSON_Template</code>        | <code>FString</code> | The HTTP request body template with placeholders for the prompt text                                                         |
| <code>Base_Prompt_Template</code> | <code>FString</code> | A structured character prompt template with placeholders for persona fields                                                  |

| Property             | Type              | Purpose                                                    |
|----------------------|-------------------|------------------------------------------------------------|
| Current_Character_ID | FString           | The active character persona (e.g., "socrates", "aspasia") |
| Characters_File_Path | FString           | Path to the JSON file containing character definitions     |
| Character_JSON_Data  | JsonObjectWrapper | Parsed character data loaded at runtime                    |
| Conversation_Context | FString           | Running context string updated with each exchange          |
| Is_Request_Pending   | bool              | Prevents concurrent API requests                           |
| Last_Response        | FString           | Cached last successful response                            |
| Last_Error           | FString           | Cached last error message                                  |

Table 5: HTTP Client Component — Configuration Properties

The component exposes two delegates:

- OnAIResponseReceived: Fires when a successful AI response is received, passing the response text to the bound NPC event.
- OnDialogueOptionsReceived: Fires when the AI returns structured dialogue options.

### 5.2.2 API Configuration

The component communicates with Google's Gemini 2.5 Flash model via HTTP POST requests. The JSON request body follows this template:

```

{
  contents: [
    {
      parts: [
        {
          text: "{prompt_text}"
        }
      ]
    }
  ]
  generationConfig: {
    temperature: 0.7
    maxOutputTokens: 1024
    topP: 0.9
  }
}

```

Figure 6: Gemini API — JSON Request Body Template

The generation configuration parameters were chosen to balance creativity with accuracy:

- Temperature (0.7): A moderate value that allows varied phrasing while avoiding hallucination of historical facts. Lower values produced repetitive responses; higher values introduced anachronistic content.
- MaxOutputTokens (1024): Limits response length to approximately 2–3 paragraphs, preventing excessively long NPC monologues that would disrupt gameplay pacing.
- TopP (0.9): Nucleus sampling that considers the top 90% probability mass, providing natural-sounding responses while filtering low-probability completions.

### 5.2.3 Character Persona System

Each NPC's AI behaviour is constrained by a character persona defined in an external JSON file (ancient\_greece\_characters.json). At runtime, the component loads and parses this file during BeginPlay via the Load\_Character\_Data function.

Each character definition contains the following fields:

| Field              | Purpose                          | Example (Socrates)                                               |
|--------------------|----------------------------------|------------------------------------------------------------------|
| CHARACTER_NAME     | The character's historical name  | Socrates                                                         |
| OCCUPATION         | Their role in the ancient world  | Philosopher                                                      |
| LOCATION           | Geographical setting             | Athens                                                           |
| ERA                | Historical time period           | 470–399 BCE                                                      |
| PERSONALITY_TRAITS | Behavioural characteristics      | Always questioning, humble about knowledge, uses irony           |
| KNOWLEDGE_AREAS    | Topics the character can discuss | Ethics, human nature, Athenian life                              |
| SPEECH_PATTERNS    | Linguistic style                 | Responds with questions, uses simple analogies, admits ignorance |

Table 6: Character Persona — JSON Field Definitions

The `Get_Character_Field` function retrieves individual fields by Character ID and field name, enabling the prompt builders to assemble persona-specific prompts dynamically.

#### 5.2.4 Prompt Construction

The component provides four prompt-building functions, each designed for a different conversational context: `Build_Full_Prompt(Character_ID, Player_Input, Context)`: Constructs a complete prompt by populating the `Base_Prompt_Template` with all character fields, the conversation context, and the player's input. The template follows this structure:

|                                                                                        |
|----------------------------------------------------------------------------------------|
| You are {CHARACTER_NAME}, a {OCCUPATION} in {LOCATION} during {ERA}.                   |
| Personality: {PERSONALITY_TRAITS}                                                      |
| Knowledge: You know about {KNOWLEDGE_AREAS} but are unaware of events after your time. |
| Speech: {SPEECH_PATTERNS}                                                              |
| Current context: {CONVERSATION_CONTEXT}                                                |
| Player said: "{PLAYER_INPUT}"                                                          |
| Respond as {CHARACTER_NAME} would, staying in character. Keep responses 2-3 sentences. |

Table 7: Base Prompt Template Structure

This template enforces several critical constraints: the character cannot reference events beyond their historical period, their personality shapes the tone of responses, and the length instruction prevents the LLM from generating walls of text.

`Build_Followup_Prompt(Character_ID, Player_Input, Context)` Used for subsequent messages in an ongoing conversation. It includes the accumulated conversation context to ensure the LLM maintains coherence across multiple exchanges.

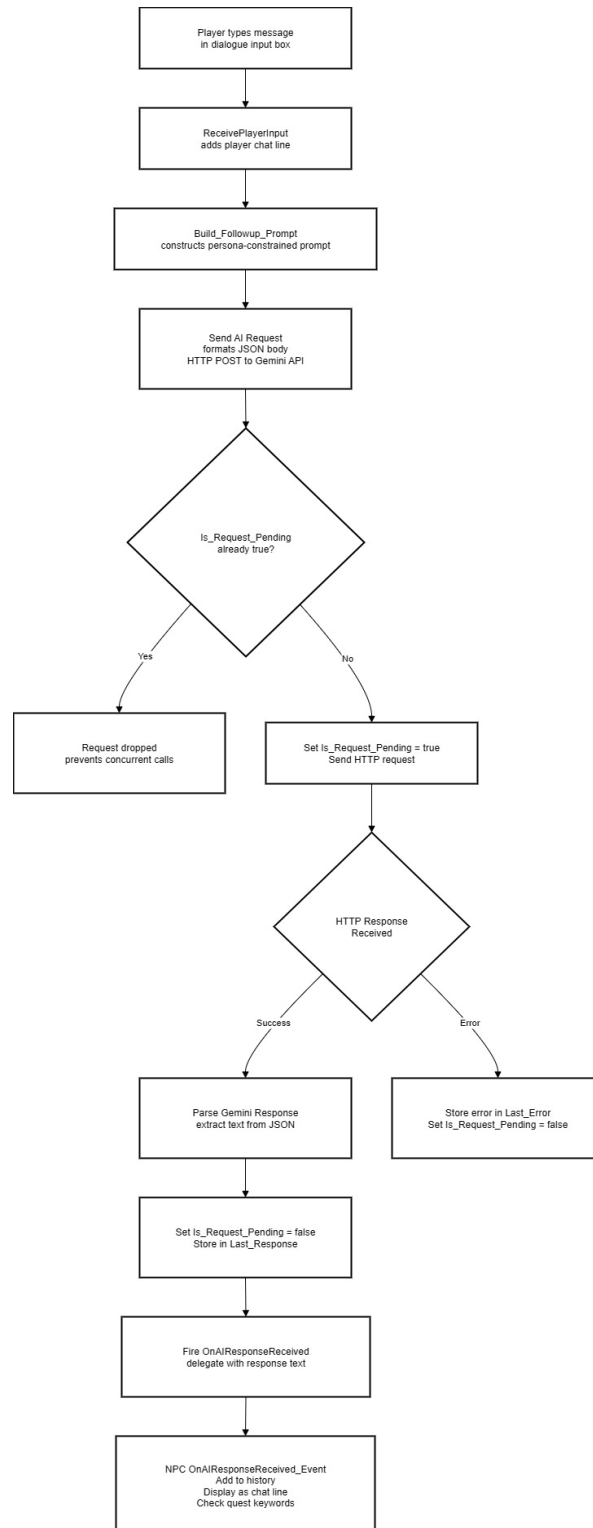


Figure 7: *Prompt Construction and AI Request Flow*

**Send AI Request:** The entry point for all AI communication. It accepts the constructed prompt, formats it into the JSON template by replacing the {0} placeholder with the prompt text, and initiates an HTTP POST request to the Gemini API endpoint with the API key appended as a query parameter. The `Is_Request_Pending` flag prevents overlapping requests

**OnRequestComplete callback:** The HTTP response callback checks `bSuccessful` to determine whether the request completed without network errors. On success, it passes the raw JSON response string to `Parse Gemini Response`.  
**Parse Gemini Response:** This function extracts the AI-generated text from the Gemini response JSON. The Gemini API returns a nested structure where the generated text is located at `candidates[0].content.parts[0].text`. The function navigates this structure and returns the extracted text string. If parsing fails, the function returns an empty string and logs the error.

### 5.2.5 Delegate Binding

The connection between the HTTP Client Component and the NPC's dialogue logic is established through delegate binding during `BeginPlay`. Each NPC's `BeginPlay` event calls the parent `BP_NPCBase` implementation, which then binds the `OnAIResponseReceived_Event` custom event to the HTTP Client Component's `OnAIResponseReceived` delegate via `AddEventToDispatcher`. This ensures that when the API returns a response, the NPC's event graph receives the response text automatically without polling or manual checking.

### 5.2.6 Historical Accuracy Constraints

A key challenge in using LLMs for educational content is ensuring historical accuracy. The persona system addresses this through multiple layers of constraint:

1. **Temporal boundary:** The prompt explicitly states that the character is "unaware of events after your time", preventing anachronistic references.
2. **Knowledge scoping:** The `KNOWLEDGE_AREAS` field limits the topics the character can authoritatively discuss.
3. **Personality enforcement:** The `SPEECH_PATTERNS` field ensures responses match the character's historical communication style - Socrates questions rather than lectures, Aspasia speaks with religious authority.
4. **Length limitation:** The instruction to "keep responses 2-3 sentences" prevents the LLM from generating lengthy passages where inaccuracies are more likely to accumulate.

Despite these constraints, AI-generated responses are not guaranteed to be historically accurate. This is why the core educational content is delivered through the scripted layer, while the AI layer serves as a supplementary engagement tool that allows curious players to explore topics in more depth.

## 5.3 Evolution of current Dialogue System:

### 5.3.1 Initial Approach - Fully AI-Driven NPCs

The original design for Chronos relied entirely on AI-generated dialogue for all NPC interactions. Every NPC response - introductions, educational content, quiz questions, and feedback — was to be generated in real time by the LLM. The player would converse freely with NPCs, and the AI would deliver historically accurate educational content organically through conversation.

This approach was abandoned during early development for three reasons:

**Unpredictable quest progression:** Tying quest objective completion to AI-generated responses made progression unreliable. The keyword detection system could miss relevant content if the LLM phrased it unexpectedly, or could falsely trigger objectives if the LLM used a keyword in an unrelated context. Players sometimes completed objectives without receiving the intended educational content, or conversely could not progress despite extensive conversation.

Cost and latency: Every NPC interaction required an API call, introducing noticeable response delays and accumulating API costs across a full playthrough. Players would experience inconsistent pacing - some responses arrived in under a second, others took several seconds, disrupting the flow of conversation.

### 5.3.2 The Hybrid Solution

These issues led to the current hybrid architecture where scripted dialogue handles all core educational content and quiz verification, while AI dialogue serves as a supplementary layer for open-ended exploration. This division ensures that the educational pipeline is deterministic and accurate, while the AI layer adds conversational depth without bearing the responsibility of delivering critical content.

### 5.3.3 Conversation History and Context Management

A key challenge in the hybrid system is maintaining conversational coherence across multiple exchanges.

The ConversationHistory array on BP\_NPCBase stores every message as an S\_ChatMessage structure containing the text and the sender role ("Player" or "AI"). This history serves two purposes:

Context for AI prompts: The conversation history is included in follow-up prompts, allowing the LLM to reference previous exchanges. Without this, each AI response would be context-free and the NPC would appear to have no memory of the conversation.

Chat repopulation: If the player closes the dialogue and returns to the same

NPC, RepopulateChatFromHistory rebuilds the chat display from the stored history, ensuring conversational continuity across multiple interaction sessions.

### 5.3.4 Token Optimisation

LLM API calls are billed per token, and prompt length directly affects both cost and response latency. Chronos implements token optimisation through two distinct prompt construction strategies:

Build\_Opening\_Prompt: Used for the first message in a conversation. It constructs a complete persona prompt with all character fields (name, occupation, location, era, personality, knowledge areas, speech patterns) but omits conversation history since none exists yet. This produces the shortest possible prompt that still establishes the character's identity and constraints.

Build\_Followup\_Prompt: Used for subsequent messages. Rather than resending the full character definition with every exchange, the follow-up prompt includes a condensed context reference and appends only the relevant conversation history. The Conversation\_Context variable on the HTTP Client Component maintains a running summary that is more token-efficient than including the raw transcript of every previous message.

This two-tier approach reduces token usage significantly over the course of a multi-turn conversation. The opening prompt establishes the full persona once, and subsequent prompts build upon that foundation without redundantly repeating the character definition. In practice, this reduced average prompt length by approximately 40% for conversations beyond three exchanges, lowering both API cost and response latency.

The maxOutputTokens parameter is set to 1024, capping response length at approximately 2-3 sentences. This prevents the LLM from generating lengthy monologues that would consume output tokens unnecessarily and disrupt gameplay pacing. The instruction "Keep responses 2-3 sentences" in the prompt template further reinforces this constraint at the model level.

## Chapter 6: Codex and Inventory Systems

### 6.1 Overview

The Codex and Inventory systems serve complementary roles in Chronos. The Codex records the player's educational progress as a persistent encyclopaedia, transforming transient dialogue into a permanent reference. The Inventory system manages physical items that the player collects and uses within the game world, bridging the gap between conversational learning and tangible interaction. Both systems are implemented as Actor Components attached to the player character, ensuring portability across level transitions.



Figure 8: Codex and Inventory Systems Overview

### 6.2 Codex System

#### 6.2.1 Purpose

The Codex exists to solve a specific educational design problem: players who engage in lengthy NPC conversations may not retain all of the information presented. Rather than requiring players to re-initiate conversations to review material, the Codex provides a categorised journal where all learned content is permanently available. Each entry corresponds to a historical topic covered during gameplay - the Parthenon, the goddess Athena, the Panathenaic festival, Socratic philosophy, Nabataean trade, and others.

#### 6.2.2 Implementation

The BP\_CodexSystem is an Actor Component with the following structure:

| Property        | Type          | Purpose                                                       |
|-----------------|---------------|---------------------------------------------------------------|
| UnlockedEntries | TArray<FName> | Array of entry IDs that the player has unlocked               |
| CodexDataTable  | UDataTable    | Reference to DT_CodexEntries containing all entry definitions |
| OnEntryUnlocked | FDelegate     | Fires when a new entry is unlocked, used for UI notifications |

Table 8: BP\_CodexSystem — Component Properties

The component provides three functions:

- `UnlockEntry(EntryID)`: Adds the given FName to the `UnlockedEntries` array if not already present, then fires the `OnEntryUnlocked` delegate. This is the only entry point for unlocking content.
- `IsEntryUnlocked(EntryID)`: Returns a boolean indicating whether a specific entry has been unlocked. Used by the UI to determine display state.
- `GetUnlockedEntries()`: Returns the full array of unlocked entry IDs. Used by the Codex journal widget to populate its display.

### 6.2.3 Entry Data

All Codex entries are defined in the `DT_CodexEntries` Data Table. Each row represents a single encyclopaedia entry containing the entry title, a detailed description of the historical topic, and categorisation metadata. Entries are authored to provide accurate, self-contained summaries that a player could reference independently of the NPC conversations that originally delivered the content.

### 6.2.4 Unlock Mechanisms

Codex entries are unlocked through two pathways:

**Direct unlock from scripted dialogue:** When a player selects a teaching dialogue option, the NPC's `ReceivePlayerInput` event explicitly calls `UnlockEntry` on the player's Codex component. The entry ID is retrieved from the NPC's `QuestToCodexMap` using the objective ID as the key. For example, completing `obj_greece_learn_temple` looks up the map and finds `codex_greece_parthenon`, which is then passed to `UnlockEntry`.

**Unlock from quiz completion:** When a player correctly answers a quiz question, the `HandleQuizAnswer` function calls `UnlockEntry` directly with the hardcoded Codex entry ID associated with that question's topic.

Both pathways converge on the same `UnlockEntry` function, ensuring consistent behaviour regardless of how the entry was earned.

### 5.2.5 Codex Entries in Chronos

The following entries are implemented in the current version of the game for Athens:

| Entry ID                                  | Topic                    | Unlocked By                                             |
|-------------------------------------------|--------------------------|---------------------------------------------------------|
| <code>codex_greece_parthenon</code>       | The Parthenon            | Learning about the temple from Aspasia                  |
| <code>codex_greece_athena</code>          | The Goddess Athena       | Learning about Athena from Aspasia                      |
| <code>codex_greece_panathenaia</code>     | The Panathenaic Festival | Learning about the festival from Aspasia                |
| <code>codex_greece_golden_owl</code>      | The Golden Owl of Athena | Learning about the owl from Aspasia or returning it     |
| <code>codex_greece_owl_theft</code>       | The Theft of the Owl     | Learning about the theft from Aspasia                   |
| <code>codex_greece_aspasia_journal</code> | Aspasia's Journal        | Completing Aspasia's introduction                       |
| <code>codex_greece_socrates</code>        | Socrates of Athens       | Completing any of Socrates' learning or quiz objectives |

Table 9: *Codex Entries — Athens Quest Line*

### 6.2.6 Codex UI

The WBP\_CodexJournal\_V2 widget provides the player-facing interface for the Codex. It is accessible via a keyboard shortcut during gameplay. The journal displays all unlocked entries in a categorised list format. Each entry can be selected to display its full description text. Locked entries are not shown, ensuring the journal only displays content the player has actually encountered.

## 6.3 Inventory System

### 6.3.1 Purpose

The Inventory system introduces a physical dimension to the educational experience. Rather than learning about artefacts solely through dialogue, players collect, carry, and use tangible items within the game world. This creates memorable interactions - finding the golden owl hidden behind Athena's statue is a more impactful learning moment than simply hearing about it from an NPC.



Figure 9: Inventory System — Item Interaction Concept

### 6.3.2 Implementation

The BP\_InventoryComponent is an Actor Component with the following structure:

| Property       | Type          | Purpose                                               |
|----------------|---------------|-------------------------------------------------------|
| CollectedItems | TArray<FName> | Array of item IDs currently in the player's inventory |

Table 10: BP\_InventoryComponent — Component Properties

The component provides four functions:

- **AddItem(ItemID):** Adds the given FName to the CollectedItems array. Called when the player picks up an item in the world.
- **RemoveItem(ItemID):** Removes the given FName from the array. Called when the player gives an item to an NPC or consumes it.
- **IsItemCollected(ItemID):** Returns a boolean indicating whether a specific item is currently in the inventory. Used by NPCs to check whether the player possesses a required item before offering certain dialogue options.
- **GetCollectedItems():** Returns the full array for UI display.

### 6.3.3 Item Actors

Items in the game world are represented by Actor blueprints that inherit from BP\_BaseItem. Each item actor defines an ItemID and implements an interaction event that adds the item to the player's inventory and completes a quest objective. The following items are implemented:

| Blueprint           | Item ID           | Location                                     | Quest Objective      |
|---------------------|-------------------|----------------------------------------------|----------------------|
| BP_GoldenOwl        | golden_owl        | Behind the statue of Athena in the Parthenon | obj_greece_find_owl  |
| BP_HoneyCake        | honey_cake        | The bazaar market in Petra                   | obj_find_cakes       |
| BP_TreasuryArtifact | treasury_artifact | Inside Al-Khazneh in Petra                   | obj_inspect_artifact |

*Table 11: Item Actor Blueprints*

When the player interacts with an item actor, the following sequence occurs:

1. The item's interaction event fires.
2. AddItem is called on the player's BP\_InventoryComponent with the item's ID.
3. UpdateObjectiveProgress is called with the corresponding quest objective ID.
4. The item actor is removed from the world or made invisible to prevent duplicate collection.

### 6.3.4 Item Usage in Dialogue

Items are consumed through NPC dialogue interactions. The dialogue system integrates with the Inventory component at two points:

**Inventory checks:** When a player selects a dialogue option that requires an item (e.g., "I found it! Here, take it back" when speaking to Aspasia about the owl), the NPC's ReceivePlayerInput event calls IsItemCollected on the player's Inventory component. If the item is present, the dialogue proceeds with the giving sequence. If not, the NPC responds with a rejection message (e.g., "You say you found it, but I see nothing in your hands").

**Item consumption:** When the item check succeeds, RemoveItem is called to remove the item from the inventory, the corresponding quest objective is completed, and the NPC's dialogue advances. For example, giving the golden owl to Aspasia removes golden\_owl from the inventory, completes obj\_greece\_return\_owl, and triggers the quest completion sequence.

### 6.3.5 Inventory UI

The WBP\_InventoryJournal widget displays the player's current inventory in a grid or list format. It reads from the BP\_InventoryComponent's CollectedItems array and displays each item with its name and description. The inventory is accessible via a keyboard shortcut during gameplay.

## 6.4 Cross-Level Persistence

Both the Codex and Inventory systems persist across level transitions through the Game Instance. When the player enters a level transition portal:

1. The level changer blueprint reads UnlockedEntries from BP\_CodexSystem and saves it to Saved\_UnlockedCodexEntries on the Game Instance.
2. It reads CollectedItems from BP\_InventoryComponent and saves it to Saved\_InventoryItems on the Game Instance.

When the new level loads, the player character's BeginPlay restores both systems:

1. It iterates through Saved\_UnlockedCodexEntries and calls UnlockEntry for each entry ID, repopulating the Codex.
2. It iterates through Saved\_InventoryItems and calls AddItem for each item ID, repopulating the inventory.

This ensures that a player who unlocks a Codex entry about the Parthenon in Athens can still reference it when they transition to Petra, and items collected in one world carry into the next.

## Chapter 7: Expansion Guide

### 7.1 Purpose

Chronos was designed with a modular architecture that allows future developers to add new historical worlds without modifying existing systems. This chapter provides a concise guide to expanding the game with new content.

### 7.2 Adding a New World

The process of adding a new historical world (e.g., Ancient Rome) requires the following steps:

1. Create the level: Create a new level asset using the same Game Mode (BP\_ThirdPersonGameMode). Design the environment and place a PlayerStart actor.
2. Create portals: Duplicate an existing portal blueprint (e.g., BP\_LevelChangerGreece) and set its Level variable to the new level. Place it in Lvl\_Void. Place an instance of BP\_LevelChangeVoid in the new level for the return journey, adding the new return objective ID (e.g., obj\_rome\_return\_void) to its UpdateObjectiveProgress calls.
3. Define quests: Add new rows to DT\_QuestList following the existing naming convention. Chain them using NextQuestID, ending with a return-to-void quest (e.g., RomeToVoidQuest) that has NextQuestID set to None.
4. Create NPCs: Create child blueprints of BP\_NPCBase. Set the ObjectivesToComplete array, DialogueKeywords map, QuestToCodexMap, QuizOptions array, and Current Character ID. Implement the Talk, ReceivePlayerInput, ShowCurrentDialogueStage, and HandleQuizAnswer events following the pattern of existing NPCs.
5. Add Codex entries: Add new rows to DT\_CodexEntries and link them to NPC objectives via each NPC's QuestToCodexMap.
6. Add items (if needed): Create child blueprints of BP\_BaseItem with unique ItemID values. Place them in the level.

### 7.3 Quick Reference

| What to Create      | Where                        | Key Properties to Set              |
|---------------------|------------------------------|------------------------------------|
| Level               | Content Browser              | Game Mode = BP_ThirdPersonGameMode |
| Portal to new world | Duplicate existing portal BP | Level = new level soft reference   |

| What to Create  | Where                                 | Key Properties to Set                                                                      |
|-----------------|---------------------------------------|--------------------------------------------------------------------------------------------|
| Return portal   | Place BP_LevelChangeVoid in new level | Add new return objective ID                                                                |
| Quests          | New rows in DT_QuestList              | Quest name, stages, objectives, NextQuestID                                                |
| NPCs            | Child BP of BP_NPCBase                | ObjectivesToComplete, DialogueKeywords, QuestToCodexMap, QuizOptions, Current Character ID |
| Codex entries   | New rows in DT_CodexEntries           | Entry title, description, category                                                         |
| Items           | Child BP of BP_BaseItem               | ItemID                                                                                     |
| Stranger update | BP_Stranger_Void BeginPlay            | Add Array_Contains check for new return quest                                              |

Table 12: Expansion Quick Reference Checklist

## 7.4 Important Notes

- No Game Instance changes required for standard expansions. The existing Saved\_ActiveQuests, Saved\_CompletedQuests, Saved\_InventoryItems, and Saved\_UnlockedCodexEntries arrays automatically accommodate new entries.
- Objective IDs must be globally unique across all quests. Follow the naming convention obj\_[world]\_[action]\_[subject] (e.g., obj\_rome\_learn\_colosseum).
- AI character personas require a new entry in the characters JSON file (ancient\_greece\_characters.json or a new file) with the character's name, era, personality, knowledge areas, and speech patterns.
- Quest chaining is automatic - once NextQuestID is set correctly in the Data Table, no additional Blueprint wiring is needed for quest flow.

## 7.5 AI NPCs in the Game:

### Petra World — Nabataean Kingdom (~1st Century AD):

| NPC                   | Role                         | Quest                   | Knowledge Areas                                                              | Speech Style                             | AI Mode  |
|-----------------------|------------------------------|-------------------------|------------------------------------------------------------------------------|------------------------------------------|----------|
| <b>Guide Abu</b>      | World Guide                  | Petra Introduction      | General Petra history, Nabataean culture, city landmarks                     | Informative, approachable, open-ended    | Fully AI |
| <b>Merchant Amina</b> | Introductory Merchant        | Welcome to Petra        | Nabataean trade, city identity, daily market life                            | Warm, welcoming, marketplace familiar    | Hybrid   |
| <b>Obodas</b>         | Caravan Master / Quest Giver | Secrets of the Merchant | Frankincense trade, tax system, Al-Khazneh, water engineering, bitumen trade | Authoritative, experienced desert trader | Hybrid   |

|               |                                |                         |                                                               |                                |        |
|---------------|--------------------------------|-------------------------|---------------------------------------------------------------|--------------------------------|--------|
| <b>Hakim</b>  | Market Merchant / Intel Source | Secrets of the Merchant | Local gossip, people's habits, NPC relationships, honey cakes | Chatty, conspiratorial, warm   | Hybrid |
| <b>Khalil</b> | Treasury Guard / Educator      | The Treasury's Secret   | Royal protocols, Treasury security, Nabataean royalty         | Gruff, duty-bound, persuadable | Hybrid |

### Athens World — Classical Athens (~5th Century BC):

| NPC             | Role                    | Quest                                | Knowledge Areas                                                 | Speech Style                                          | AI Mode |
|-----------------|-------------------------|--------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------|---------|
| <b>Aspasia</b>  | Priestess / Quest Giver | Welcome to Athens, Return of the Owl | Parthenon, goddess Athena, Panathenaic festival, the golden owl | Solemn, religious authority                           | Hybrid  |
| <b>Socrates</b> | Philosopher / Educator  | The Wisdom of Socrates               | Ethics, nature of knowledge, Kleon, Socratic trial              | Questions rather than answers, uses irony and analogy | Hybrid  |

## Chapter 8 Testing Phase:

Testing was conducted iteratively throughout development, with a focus on two critical areas: the AI integration pipeline and end-to-end gameplay flow. This chapter documents the testing process, key bugs discovered during playtesting, and an evaluation of the game against its stated objectives.

### 8.1 AI Integration Testing via Postman

Before integrating the Gemini API into the game, the HTTP client pipeline was tested independently using Postman. This allowed the API request format, response parsing, and character persona prompts to be validated without the overhead of launching the game.

The Gemini 2.5 Flash endpoint was tested with raw POST requests to confirm connectivity, authentication via the API key, and correct response structure. The JSON request body was iterated upon in Postman until the response format was stable and parseable. The response structure candidates[0].content.parts[0].text was confirmed and used to build the Parse Gemini Response function in the HTTP Client Component.

#### 8.1.1 Character Persona Testing

Each character persona was tested in Postman by sending the fully constructed prompt (using the Base\_Prompt\_Template populated with character fields) and evaluating the responses against the following criteria:

| Criteria                | Test Method                                                                                      |
|-------------------------|--------------------------------------------------------------------------------------------------|
| Historical accuracy     | Verified key claims against known historical facts                                               |
| Temporal boundary       | Asked about events after the character's era to confirm the model refused or expressed ignorance |
| Personality consistency | Sent identical questions to different personas and verified distinct response styles             |
| Response length         | Confirmed responses stayed within the 2-3 sentence instruction                                   |

| Criteria        | Test Method                                                                                  |
|-----------------|----------------------------------------------------------------------------------------------|
| Character voice | Verified Socrates responded with questions and irony, Aspasia spoke with religious authority |

*Table 13: AI Character Persona — Testing Criteria*

Example test: Sending "What do you think about Socrates' philosophy?" to the Socrates persona correctly returned a response expressing ignorance of Aristotle, as the prompt constrains Socrates to knowledge prior to 399 BCE.

## Chapter 9: Conclusion and Future Work

### 9.1 Summary

The development of Chronos provides concrete evidence that AI-integrated NPCs are not merely a theoretical possibility but a practical, implementable feature within modern game engines. Through this project, a fully functional AI dialogue pipeline was built and deployed within Unreal Engine 5 - connecting a large language model to in-game NPC actors that respond dynamically to arbitrary player input while maintaining historically accurate personas, contributing to quest progression through keyword detection, and operating alongside scripted content without disrupting the game's pedagogical structure.

The project demonstrates that the technical barriers to AI NPC integration are surmountable. The HTTP Client Component developed for Chronos communicates with Google's Gemini 2.5 Flash model using standard HTTP POST requests, parses JSON responses, and delivers them through the existing dialogue widget - all without requiring custom C++ code, engine modifications, or specialised middleware. The entire integration was achieved within Unreal Engine's Blueprint visual scripting system, making it accessible to developers without deep programming expertise.

Critically, this project also demonstrates that AI-driven dialogue and structured educational design are not mutually exclusive. The hybrid architecture - where scripted dialogue delivers verified content and AI dialogue extends the experience - proves that LLMs can enhance educational games without replacing the authored content that guarantees accuracy. The persona system, conversation history, and keyword detection mechanisms show that AI responses can be meaningfully integrated into game systems rather than existing as isolated chat windows. The implications extend beyond educational games. The architecture implemented in Chronos - a character persona template constraining an LLM, delegate-bound response handling, and keyword-to-objective mapping - is transferable to any genre where NPC conversation depth matters. RPGs, adventure games, training simulations, and interactive fiction could all benefit from the same hybrid approach demonstrated here.

### 9.2 Limitations

**AI accuracy:** The persona system constrains LLM responses to a historical period and personality but cannot guarantee complete factual accuracy. Core educational content is delivered through the scripted layer to mitigate this, but occasional inaccuracies may appear in AI-generated free-form responses.

**Assessment depth:** Multiple-choice quizzes assess recognition rather than deep understanding. A player may select the correct answer through elimination rather than genuine comprehension.

**Audio content:** Placeholder audio recordings are used for NPC dialogue. Professionally recorded voice acting would significantly enhance the sense of immersion and historical authenticity.

Content breadth: Two historical worlds effectively demonstrate the concept, but a broader range of civilisations would strengthen the game's value as a comprehensive educational tool.

### 9.3 Future Work

**Additional historical worlds:** The modular architecture and expansion guide (Chapter 7) make adding new civilisations straightforward. Candidate worlds include Ancient Rome, Medieval Cairo, and Dynastic China - each offering rich educational content suited to the existing Teach–Test–Advance framework.

**AI-powered open-ended assessment:** Replacing multiple-choice quizzes with free-text responses evaluated by the LLM would assess genuine understanding rather than recognition. The existing AI pipeline could be extended to compare player explanations against expected historical facts and score them accordingly.

**Classroom integration:** A teacher-facing dashboard for monitoring student progress through Codex completion rates, Knowledge Points scores, and quest advancement would extend the game's applicability in formal educational settings.

**Voice synthesis:** Integrating text-to-speech with character-appropriate voices for AI-generated responses would eliminate the current distinction between voiced scripted dialogue and text-only AI responses.

## References:

- [1] Ubisoft Montpellier. (2014). Valiant Hearts: The Great War [Video game]. Ubisoft.
- [2] Ubisoft. (2018). Assassin's Creed Origins: Discovery Tour [Video game]. Ubisoft.
- [3] Mateas, M., & Stern, A. (2005). Structuring content in the Façade interactive drama architecture. Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 1(1), 93–98.  
<https://doi.org/10.1609/aiide.v1i1.18722>
- [4] Rowe, J. P., Shores, L. R., Mott, B. W., & Lester, J. C. (2011). Integrating learning, problem solving, and engagement in narrative-centered learning environments. International Journal of Artificial Intelligence in Education, 21(1–2), 115–133.
- [5] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>
- [6] NVIDIA. (2023). NVIDIA Avatar Cloud Engine (ACE) for games. NVIDIA Newsroom.  
<https://nvidianews.nvidia.com/news/nvidia-ace-for-games-sparks-life-into-virtual-characters-with-generative-ai>
- [7] Inworld AI. (2023). Character Engine: AI-powered NPCs for games. Inworld AI. <https://inworld.ai>
- [8] Proxima Beta. (2024). Suck Up! [Video game]. Steam.  
[https://store.steampowered.com/app/2726370/Suck\\_Up/](https://store.steampowered.com/app/2726370/Suck_Up/)
- [9] Epic Games. (2022). Unreal Engine 5. Epic Games. <https://www.unrealengine.com>
- [10] Google. (2025). Gemini 2.0 Flash. Google AI for Developers. <https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash>

## Appendix:

### A: Asset Credits

The following third-party assets were used in the development of Chronos. All assets were used in accordance with their respective licences.

#### 3D Environment Assets

Table 15: 3D Environment Asset Credits

| Asset                                                | Source    | Author                                                 | Usage                   |
|------------------------------------------------------|-----------|--------------------------------------------------------|-------------------------|
| Petra Environment (Treasury, Siq, bazaar structures) | Sketchfab | <a href="#">Global Digital Heritage and GDH-Afrika</a> | Petra world environment |
| Greece Portal                                        | Sketchfab | <a href="#">Ancient greek kiosk</a>                    | Portal mesh in Void hub |
| Jordan Portal                                        | Sketchfab | <a href="#">JordanPortal</a>                           | Portal mesh in Void hub |

#### Character Assets

Table 16: Character Asset Credits

| Asset              | Source                         | Usage                                                                |
|--------------------|--------------------------------|----------------------------------------------------------------------|
| NPCs               | Tripo3dai (Generated using AI) | NPC characters ( <a href="#">Aspasia</a> , Socrates, and other NPCs) |
| Survival Character | Fab Marketplace (free)         | Player character mesh                                                |
| RPG Character      | Fab Marketplace                | Additional character animations                                      |

#### Other Assets

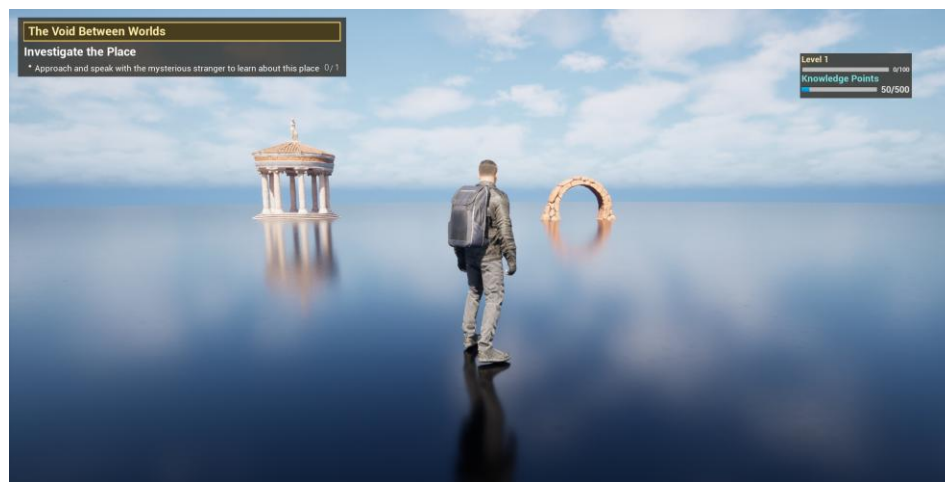
Table 17: Other Asset Credits

| Asset                                         | Source                     | Usage                                |
|-----------------------------------------------|----------------------------|--------------------------------------|
| Engine default assets (BasicShapes, Sky, Fog) | Epic Games / Unreal Engine | Void hub environment, basic geometry |

### B: Game Screenshots



Figure 10: Game Screenshots — Title and Menu Screens



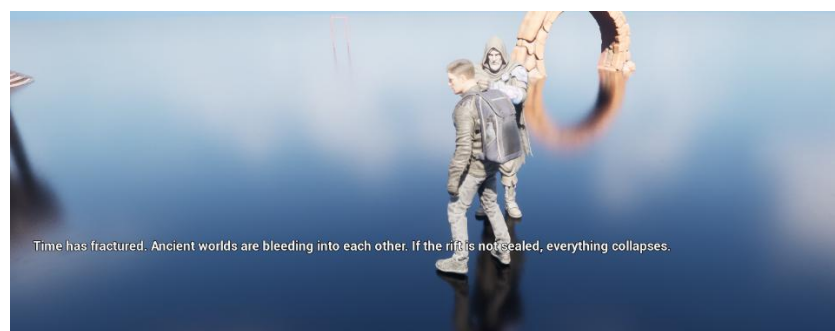
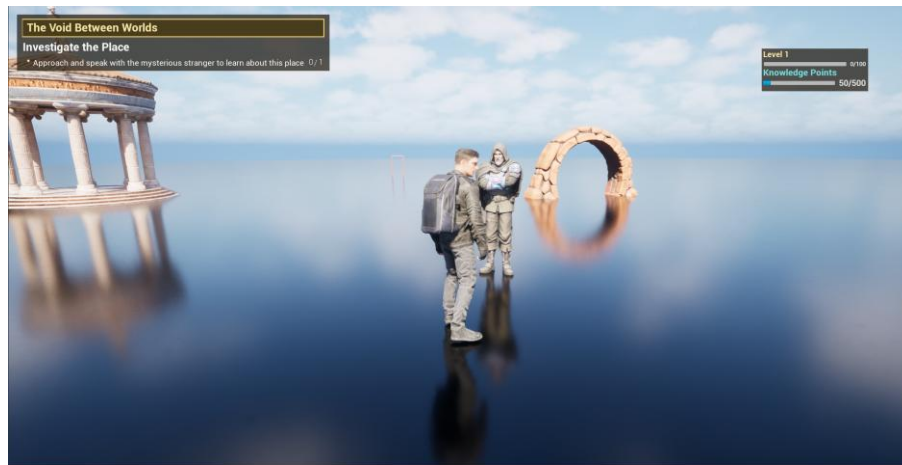


Figure 11: Void — World Environments

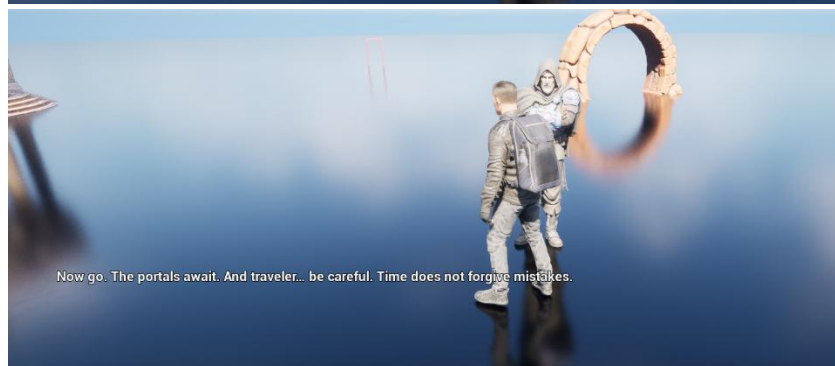
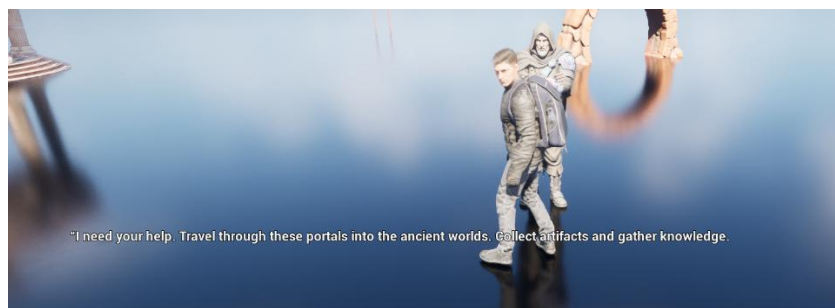


Figure 12: Game Screenshots — UI and Gameplay (continued)

## B.1 Petra World:



Figure 13: Petra World — Environment



Figure 14: Petra World — Al-Khazneh



Figure 15: Petra World — Merchant Amina



Figure 16: Petra World — Amina Teaching Phase

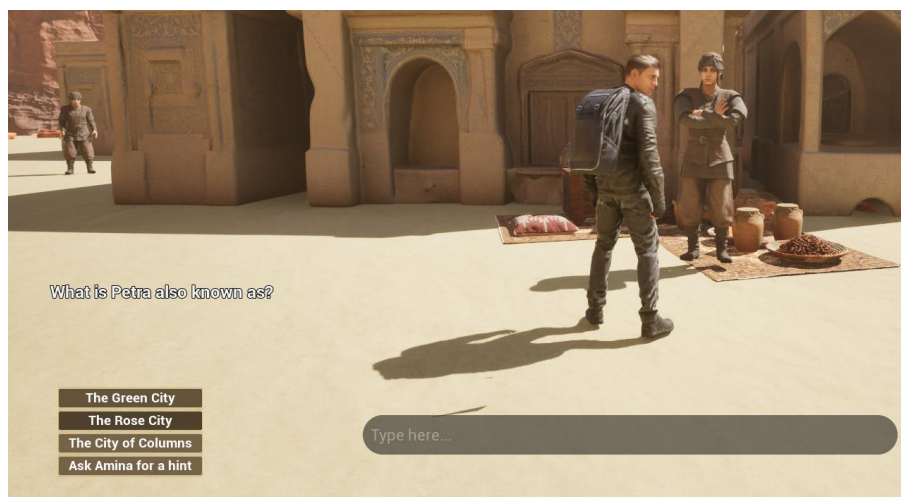




Figure 17: Petra World — Amina AI Response



Figure 18: Petra World — KP notifications



Figure 18: Petra World — Obodas

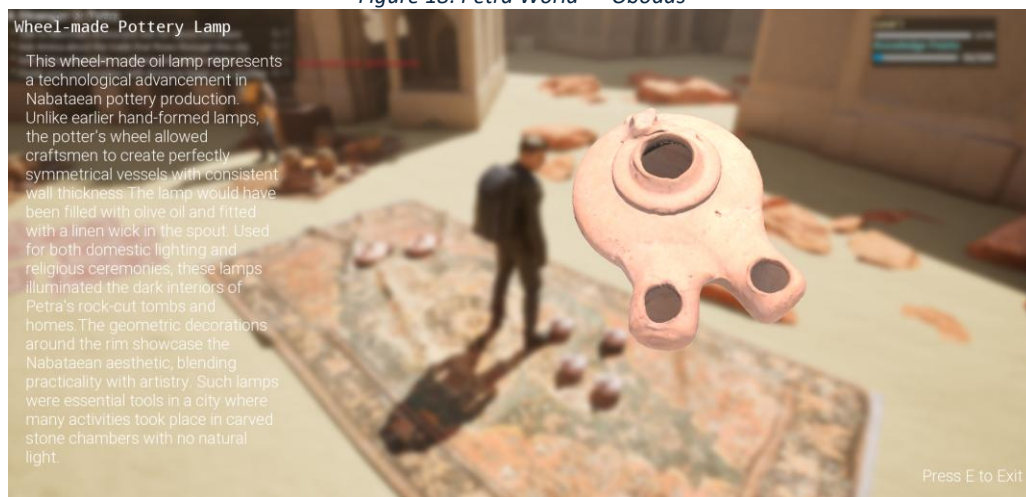


Figure 19: Inspect Item Feature





*Figure 20: Khalil Stopping the Player*



*Figure 21: Petra World — Khalil*



Figure 22: Petra World — Talking to Khalil



Figure 23: Petra World — Talking to Khalil Continued



Figure 24: Petra World — Treasury Artifact



Figure 25: Petra World — Complete Treasury Mission

## B.2 Athens World:



Figure 26: Athens World — Parthenon



Figure 27: Athens World – Aspasia

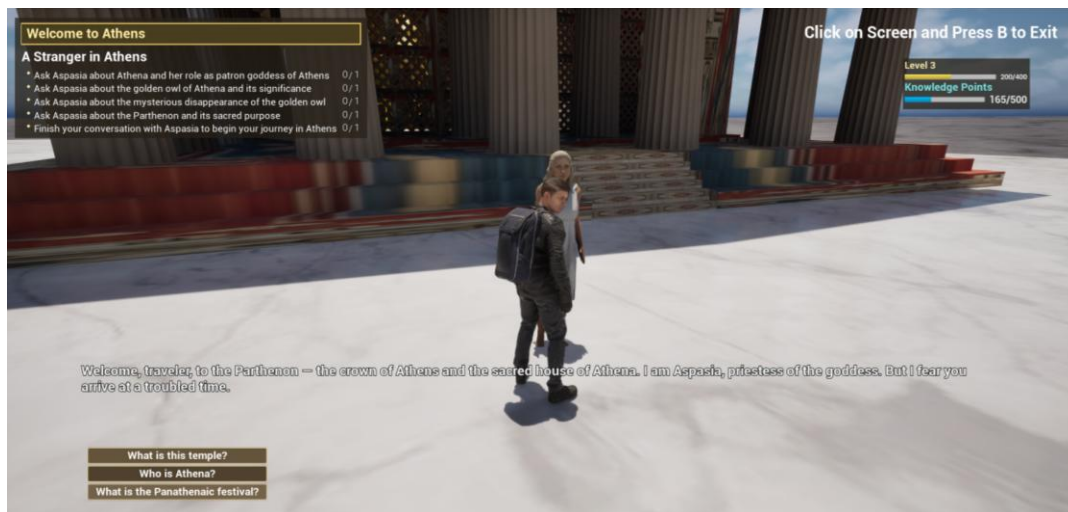


Figure 26: Athens World — Aspasia Teaching Phase

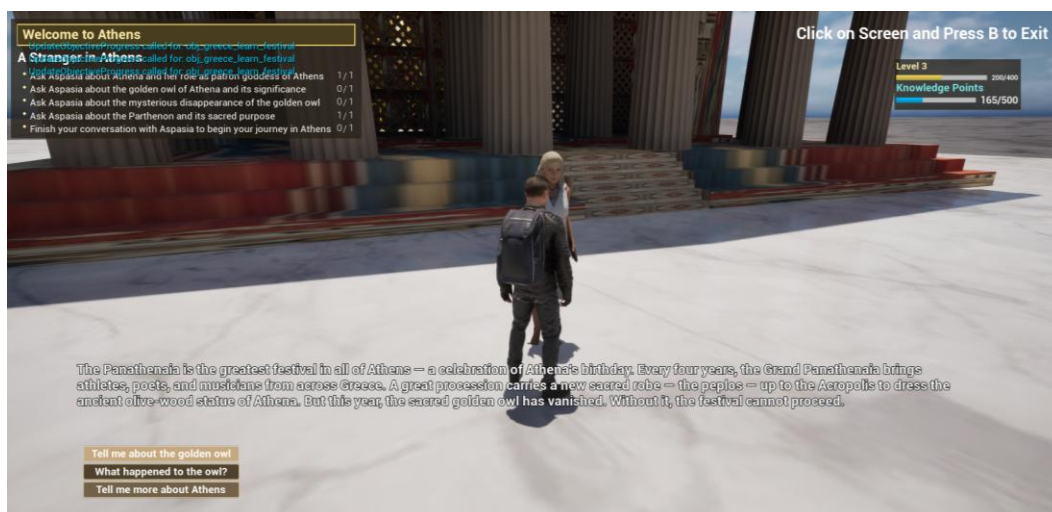


Figure 27: Athens World — Talking to Aspasia

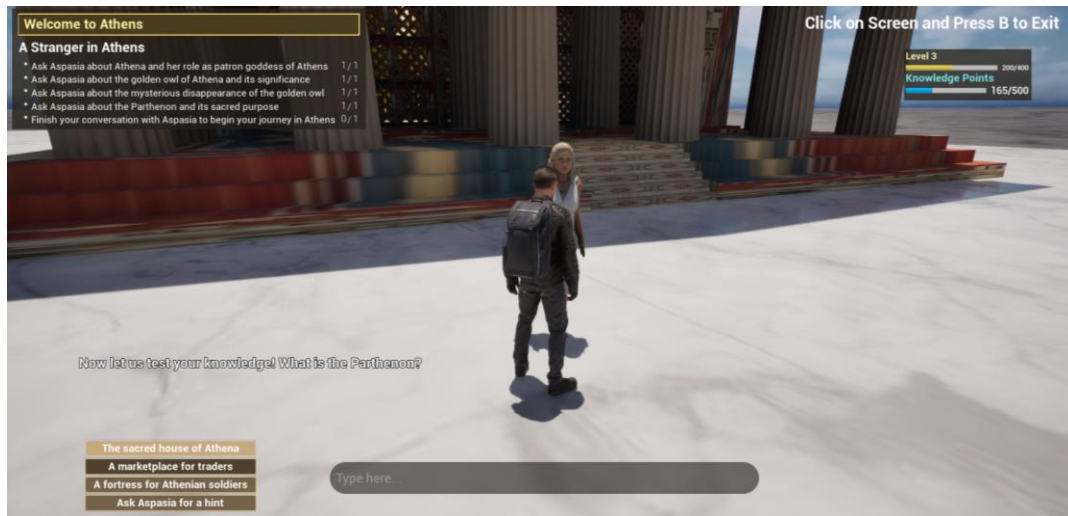


Figure 28: Athens World — Aspasia Quiz



Figure 29: Athens World — Completing mission



Figure 30: Athens World — Socrates



Figure 31: Athens World — Talking to Socrates



Figure 32: Athens World — Socrates Quiz



Figure 33: Athens World - AI response from Socrates



Figure 34: Athens World – Statue of Athena inside Parthenon



Figure 35: Athens World – Collecting the Golden Owl



Figure 36: Athens World – Returning the Golden Owl

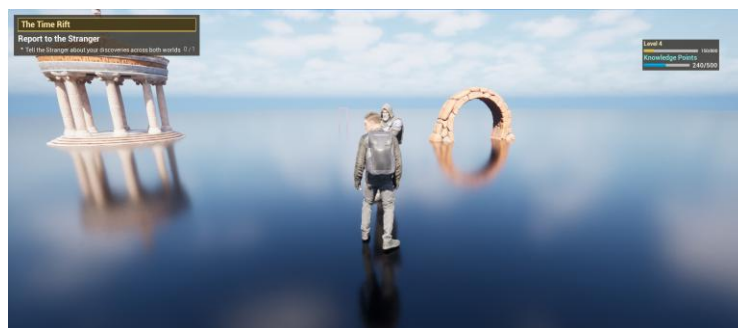


Figure 37: Void Final Mission to Complete the Game

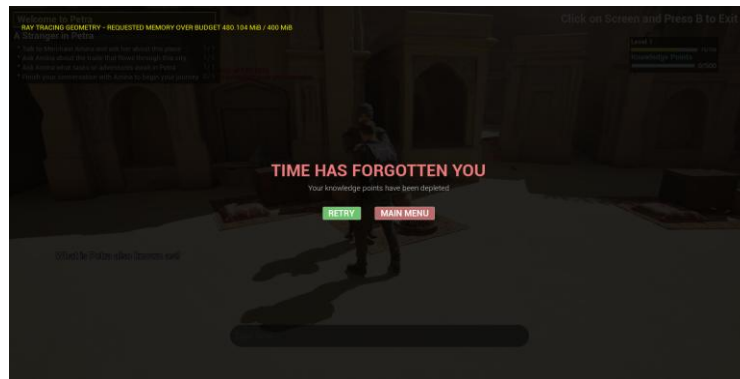


Figure 38: Game Over

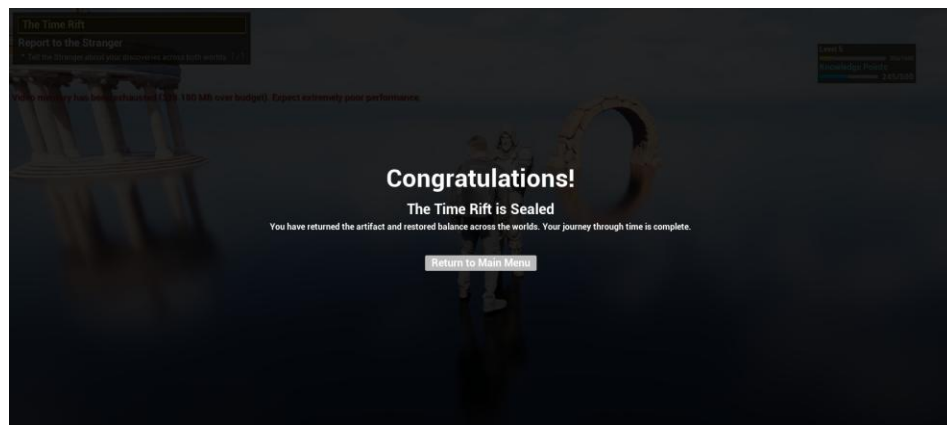


Figure 39: Void – Game Complete Window